
Documentation: process_monitor.py

Overall Purpose: A Dual-Role Process Guard

This module acts like a security agent for all the programs (processes) running on the server. It performs two very different but equally important jobs.

Job 1: The Roving Guard (Generic Threat Hunting)

- This part of the monitor watches for **any new process** that starts on the system.
- As soon as a new process appears, the Roving Guard quickly inspects its **command line** to see if it matches a list of known suspicious commands (like whoami, ncat, etc.) from the config.json.
- This is designed to catch unexpected malicious activity from an attacker who has gained some access to the machine.

Job 2: The Bodyguard (Specific Process Health)

- This part of the monitor is assigned to protect specific, important applications, which we'll call **VIPs** (Very Important Processes).
- The config.json file tells the monitor which processes are VIPs (for example, the main node process for Juice Shop).
- The Bodyguard's only job is to constantly check that its assigned VIP is still running correctly. If the VIP process crashes or disappears, the Bodyguard immediately raises an alert.

Code Breakdown

ProcessMonitor Class

`__init__(self, ...)` (The Constructor)

- It performs the standard BaseMonitor setup.
- `self.known_pids = set()`: This is the "Roving Guard's" memory. It's a set that will store all the Process IDs (PIDs) it saw during its last check.
- `self.monitored_procs = {}`: This is the "Bodyguard's" memory. It's a dictionary that will store the details (like the PID and name) of the specific VIP process it is currently protecting.
- `self.suspicious_command_patterns = [...]`: It pre-compiles the list of suspicious commands from the config file into regular expressions to make searching faster.

`run(self)`

This is the main loop that executes every few seconds. In each run, it performs both of its jobs.

Part 1: The Roving Guard's Patrol

1. It gets a fresh "snapshot" of all currently running PIDs using `psutil.pids()`.
2. It uses set math (`current_pids - self.known_pids`) to instantly find only the `new_pids`—processes that have started since the last check.
3. It loops through **only these new processes**. For each one, it gets more details and inspects its command line.
4. If the command line matches any of the `suspicious_command_patterns`, it logs a PROCESS alert.
5. **Active Response Example:** There is a special, high-priority check for a **reverse shell** (`ncat -e /bin/bash`). A reverse shell is a powerful attack tool that gives the attacker a command prompt on your server. Because this is so dangerous, the monitor doesn't just

log it; it includes "action": "kill_process" in the alert. This is a direct request to the main EDRAgent to terminate the malicious process immediately.

Part 2: The Bodyguard's Check-in

1. After checking for new threats, it calls the helper method `self._check_specific_processes()` to perform its second job.

Part 3: Updating Memory

1. At the very end, it updates the Roving Guard's memory for the next cycle:
`self.known_pids = current_pids.`

`_check_specific_processes(self)`

This helper method contains all the logic for the "Bodyguard" role. It performs its check in two phases.

Phase 1: Is our current VIP still healthy?

1. It gets a list of all currently running processes and their information.
2. It looks at the VIPs it's already tracking in `self.monitored_procs`.
3. For each one, it asks: "Is the PID I was tracking still running? And is its name still the same?"
4. If the process has disappeared, it logs a PROCESS-HEALTH alert and removes the process from its tracking list, since it's no longer there to be monitored.

Phase 2: Do we need to start guarding a new VIP?

1. It checks the processes rules in the config file.
2. For each rule, it checks if it is *already* tracking a process for that rule.
3. If not, it scans the complete list of running processes to find one that matches the rule's criteria (e.g., name is node and command line contains `/home/james/juice-shop/build/app.js`).
4. If it finds a match, it adds the process to its `self.monitored_procs` dictionary and logs an informational alert saying it has now started tracking this important process.